main.py

```
import json
import tkinter as tk
from tkinter import ttk, messagebox
from pathlib import Path
import uuid
import requests
from requests.exceptions import RequestException
DATA_FILE = Path(__file__).with_name("media.json")
class LibraryApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Atharva Rakshe Library")
        self.root.geometry("900x600")
        self.data = []
        self.sort_ascending = True
        # Configure Treeview style with alternating row colors
        style = ttk.Style()
        style.theme_use('clam')
        style.configure("Treeview", rowheight=28, font=("Arial", 10))
        style.configure("Treeview.Heading", font=("Arial", 11, "bold"))
        style.configure("Treeview", background="#F5F5F5", fieldbackground="#F5F5F5")
        style.map('Treeview', background=[('selected', '#5C4A99')])
        # Header frame
        header = tk.Frame(root, bg="#5C4A99", height=80)
        header.pack(side=tk.TOP, fill=tk.X)
        header.pack_propagate(False)
        title_label = tk.Label(header, text="■  Scholar's Digital Library", font=("Arial", 2
4, "bold"), fg="white", bg="#5C4A99")
        title_label.pack(pady=(10, 0))
        subtitle_label = tk.Label(header, text="Organize and manage your media collection",
font=("Arial", 10), fg="#E0E0E0", bg="#5C4A99")
        subtitle_label.pack(pady=(0, 10))
        # Top frame: category, load, search
        top = ttk.Frame(root, padding=(8, 6))
        top.pack(side=tk.TOP, fill=tk.X)
        ttk.Label(top, text="Category:").pack(side=tk.LEFT)
        self.category_var = tk.StringVar()
        self.category_cb = ttk.Combobox(top, textvariable=self.category_var, width=18, state
='readonly')
        self.category_cb.pack(side=tk.LEFT, padx=(6, 8))
        # Bind selection event to immediately filter by category when user selects one
        self.category_cb.bind('<<ComboboxSelected>>', lambda e: self.on_category_change())
        self.load_btn = ttk.Button(top, text="Load", command=self.on_load)
        self.load_btn.pack(side=tk.LEFT)
        ttk.Label(top, text="  Name:").pack(side=tk.LEFT, padx=(10, 4))
        self.search_var = tk.StringVar()
        self.search_entry = ttk.Entry(top, textvariable=self.search_var, width=30)
        self.search_entry.pack(side=tk.LEFT)
        self.search_btn = ttk.Button(top, text="Search", command=self.on_search)
        self.search_btn.pack(side=tk.LEFT, padx=(6, 8))
        # Action buttons
        self.erase_btn = ttk.Button(top, text="Erase", command=self.on_erase)
        self.erase_btn.pack(side=tk.RIGHT, padx=(6, 0))
        self.new_btn = ttk.Button(top, text="New", command=self.on_new)
        self.new_btn.pack(side=tk.RIGHT, padx=(6, 0))
        self.sort_btn = ttk.Button(top, text="Sort by Date ↑", command=self.on_sort_toggle)
        self.sort_btn.pack(side=tk.RIGHT, padx=(6, 14))
        # Treeview container frame
        tree_container = tk.Frame(root)
        tree_container.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=8, pady=(6, 8))
        # Treeview (with serial number column)
        columns = ("sno", "name", "author", "date", "category")
        self.tree = ttk.Treeview(tree_container, columns=columns, show='headings', style="Tr
eeview")
        # Headings: serial number + book fields
        self.tree.heading("sno", text="S.No")
        self.tree.heading("name", text="Name")
        self.tree.heading("author", text="Author")
        self.tree.heading("date", text="Date")
```

```
        self.tree.heading("category", text="Category")
        # Column sizes and alignment
        self.tree.column("sno", anchor='center', width=60)
        self.tree.column("name", anchor='center', width=360)
        self.tree.column("author", anchor='center', width=160)
        self.tree.column("date", anchor='center', width=80)
        self.tree.column("category", anchor='center', width=120)
        # Bind tags to rows for alternating colors
        self.tree.tag_configure('oddrow', background='#FFFFFF')
        self.tree.tag_configure('evenrow', background='#E8F4F8')
        vsb = ttk.Scrollbar(tree_container, orient=tk.VERTICAL, command=self.tree.yview)
        self.tree.configure(yscroll=vsb.set)
        vsb.pack(side=tk.RIGHT, fill=tk.Y)
        self.tree.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        # Empty state label
        self.empty_label = tk.Label(tree_container, text="Add book to the library", font=("A
rial", 14), fg="#999999")
        self.empty_label.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        # Context menu for right-click actions (Edit / Cancel)
        self.context_menu = tk.Menu(self.root, tearoff=0)
        self.context_menu.add_command(label="Edit", command=self.on_edit_selected)
        self.context_menu.add_command(label="Cancel", command=lambda: None)
        # Bind right-click on tree to show context menu
        self.tree.bind('<Button-3>', self.on_right_click)
        # load initial
        self.on_load()
    def read_json(self):
        if not DATA_FILE.exists():
            return []
        try:
            with DATA_FILE.open('r', encoding='utf-8') as f:
                return json.load(f)
        except Exception:
            messagebox.showerror("Error", f"Failed to read {DATA_FILE}")
            return []
    def write_json(self):
        try:
            with DATA_FILE.open('w', encoding='utf-8') as f:
                json.dump(self.data, f, indent=2, ensure_ascii=False)
        except Exception:
            messagebox.showerror("Error", f"Failed to write {DATA_FILE}")
    def on_load(self):
        # Prefer backend if available
        try:
            resp = requests.get('http://127.0.0.1:5000/media', timeout=1)
            if resp.status_code == 200:
                self.data = resp.json()
            else:
                self.data = self.read_json()
        except RequestException:
            self.data = self.read_json()
        # Ensure every book has a unique id; if missing, create one and persist
        updated = False
        for b in self.data:
            if 'id' not in b:
                b['id'] = str(uuid.uuid4())
                updated = True
        if updated:
            # persist locally if using local JSON
            try:
                self.write_json()
            except Exception:
                pass
        self.populate_categories()
        self.refresh_treeview()
    def populate_categories(self):
        # Gather categories from the data; ensure we always have an "All" option
        cats = sorted({item.get('category','') for item in self.data if item.get('category')
})
        values = ["All"] + cats
        self.category_cb['values'] = values
        # If a category was previously selected, try to keep it; otherwise default to All
```

```
        current = self.category_var.get()
        if current and current in values:
            self.category_cb.set(current)
        else:
            self.category_cb.set("All")
        # If there are no categories (empty dataset), make sure combobox still has All
        if not cats:
            self.category_cb['values'] = ["All"]
            self.category_cb.set("All")
    def refresh_treeview(self, category=None, name_filter=None):
        for r in self.tree.get_children():
            self.tree.delete(r)
        # If no explicit category provided, use the currently selected category in the combo
box
        if category is None:
            category = self.category_var.get() or "All"
        filtered = self.data
        if category and category != "All":
            filtered = [b for b in filtered if b.get('category') == category]
        if name_filter:
            q = name_filter.strip().lower()
            filtered = [b for b in filtered if q in b.get('name','').lower()]
        # sort by date
        def get_year(b):
            try:
                return int(b.get('date') or 0)
            except Exception:
                return 0
        filtered = sorted(filtered, key=get_year, reverse=not self.sort_ascending)
        for idx, b in enumerate(filtered):
            # Serial number should reflect the visible row index (1-based)
            serial = idx + 1
            tag = 'evenrow' if idx % 2 == 0 else 'oddrow'
            # Insert with serial number as first column and use book id as item iid
            item_id = b.get('id') or str(uuid.uuid4())
            # Ensure id present in data
            if 'id' not in b:
                b['id'] = item_id
            self.tree.insert('', tk.END, iid=item_id, values=(serial, b.get('name',''), b.ge
t('author',''), b.get('date',''), b.get('category','')), tags=(tag,))
        # Show/hide empty state label
        if not filtered:
            self.empty_label.pack(side=tk.TOP, fill=tk.BOTH, expand=True)
        else:
            self.empty_label.pack_forget()
    def on_search(self):
        cat = self.category_var.get()
        name = self.search_var.get()
        self.refresh_treeview(category=cat, name_filter=name)
    def on_category_change(self):
        """Called when the combobox selection changes; refresh table to show only that categ
ory."""
        cat = self.category_var.get() or "All"
        self.refresh_treeview(category=cat, name_filter=self.search_var.get())
    def on_new(self):
        NewBookWindow(self)
    def on_right_click(self, event):
        """Show context menu when user right-clicks a row."""
        # Identify the row under the mouse pointer
        iid = self.tree.identify_row(event.y)
        if iid:
            # Select the row so actions apply to it
            self.tree.selection_set(iid)
            try:
                self.context_menu.tk_popup(event.x_root, event.y_root)
            finally:
                self.context_menu.grab_release()
    def on_edit_selected(self):
        """Called when Edit is chosen from the context menu; opens edit dialog."""
        sel = self.tree.selection()
        if not sel:
            messagebox.showwarning("No selection", "Please select a book to edit")
```

```
            return
        # Use the tree item's iid (which we set to the book's unique id) to find the book
        iid = sel[0]
        idx = None
        for i, b in enumerate(self.data):
            if str(b.get('id')) == str(iid):
                idx = i
                break
        if idx is None:
            messagebox.showerror("Not found", "Selected book was not found in data")
            return
        EditBookWindow(self, idx)
    def on_erase(self):
        sel = self.tree.selection()
        if not sel:
            messagebox.showwarning("No selection", "Please select a book to delete")
            return
        # Find the selected item's id (iid)
        iid = sel[0]
        # Find index in data by id
        to_remove = None
        for i, b in enumerate(self.data):
            if str(b.get('id')) == str(iid):
                to_remove = i
                name = b.get('name')
                break
        if not messagebox.askyesno("Confirm", f"Delete '{name}'?"):
            return
        if to_remove is not None:
            deleted = None
            # try backend delete first
            try:
                resp = requests.delete(f'http://127.0.0.1:5000/media/{iid}', timeout=1)
                if resp.status_code == 200:
                    deleted = True
            except RequestException:
                deleted = None
            if deleted:
                # reload from backend
                self.on_load()
            else:
                # fallback to local deletion
                del self.data[to_remove]
                self.write_json()
                self.on_load()
    def on_sort_toggle(self):
        self.sort_ascending = not self.sort_ascending
        arrow = '↑' if self.sort_ascending else '↓'
        self.sort_btn.config(text=f"Sort by Date {arrow}")
        self.refresh_treeview(category=self.category_var.get(), name_filter=self.search_var.
get())
class NewBookWindow:
    def __init__(self, app: LibraryApp):
        self.app = app
        self.top = tk.Toplevel(app.root)
        self.top.title("Add New Book")
        self.top.geometry("420x320")
        frm = ttk.Frame(self.top, padding=12)
        frm.pack(fill=tk.BOTH, expand=True)
        ttk.Label(frm, text="Name").grid(row=0, column=0, sticky='w')
        self.name_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.name_var, width=48).grid(row=0, column=1, pady=6)
        ttk.Label(frm, text="Author").grid(row=1, column=0, sticky='w')
        self.author_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.author_var, width=48).grid(row=1, column=1, pady=6)
        ttk.Label(frm, text="Date (year)").grid(row=2, column=0, sticky='w')
        self.date_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.date_var, width=20).grid(row=2, column=1, sticky='w
', pady=6)
        ttk.Label(frm, text="Category").grid(row=3, column=0, sticky='w', pady=6)
        self.cat_var = tk.StringVar()
        cats = ["Book", "Film", "Magazine"]
```

```
        self.cat_cb = ttk.Combobox(frm, textvariable=self.cat_var, values=cats, width=45)
        self.cat_cb.grid(row=3, column=1, sticky='ew', pady=6)
        self.cat_cb.current(0)
        save_btn = ttk.Button(frm, text="Save", command=self.on_save)
        save_btn.grid(row=4, column=1, pady=(18, 0))
    def on_save(self):
        name = self.name_var.get().strip()
        author = self.author_var.get().strip()
        date = self.date_var.get().strip()
        cat = self.cat_var.get().strip() or 'Uncategorized'
        if not name:
            messagebox.showwarning("Validation", "Name is required")
            return
        # Try to create via backend if available
        try:
            resp = requests.post('http://127.0.0.1:5000/media', json={
                'name': name, 'author': author, 'date': date, 'category': cat
            }, timeout=1)
            if resp.status_code in (200, 201):
                self.app.on_load()
                self.top.destroy()
                return
        except RequestException:
            pass
        # Fallback to local create
        new = {"id": str(uuid.uuid4()), "name": name, "author": author, "date": date, "categ
ory": cat}
        self.app.data.append(new)
        self.app.write_json()
        self.app.on_load()
        self.top.destroy()
class EditBookWindow:
    def __init__(self, app: LibraryApp, index: int):
        self.app = app
        self.index = index
        self.top = tk.Toplevel(app.root)
        self.top.title("Edit Book")
        self.top.geometry("420x320")
        frm = ttk.Frame(self.top, padding=12)
        frm.pack(fill=tk.BOTH, expand=True)
        # Load current book data
        book = app.data[index]
        ttk.Label(frm, text="Name").grid(row=0, column=0, sticky='w')
        self.name_var = tk.StringVar(value=book.get('name',''))
        ttk.Entry(frm, textvariable=self.name_var, width=48).grid(row=0, column=1, pady=6)
        ttk.Label(frm, text="Author").grid(row=1, column=0, sticky='w')
        self.author_var = tk.StringVar(value=book.get('author',''))
        ttk.Entry(frm, textvariable=self.author_var, width=48).grid(row=1, column=1, pady=6)
        ttk.Label(frm, text="Date (year)").grid(row=2, column=0, sticky='w')
        self.date_var = tk.StringVar(value=book.get('date',''))
        ttk.Entry(frm, textvariable=self.date_var, width=20).grid(row=2, column=1, sticky='w
', pady=6)
        ttk.Label(frm, text="Category").grid(row=3, column=0, sticky='w', pady=6)
        self.cat_var = tk.StringVar(value=book.get('category','Uncategorized'))
        cats = ["Book", "Film", "Magazine"]
        self.cat_cb = ttk.Combobox(frm, textvariable=self.cat_var, values=cats, width=45)
        self.cat_cb.grid(row=3, column=1, sticky='ew', pady=6)
        # if current category is not in list, allow it by setting value
        if self.cat_var.get() not in cats:
            self.cat_cb.set(self.cat_var.get())
        else:
            self.cat_cb.current(cats.index(self.cat_var.get()))
        save_btn = ttk.Button(frm, text="Save", command=self.on_save)
        save_btn.grid(row=4, column=1, pady=(18, 0))
    def on_save(self):
        # Update the book at self.index with new values
        name = self.name_var.get().strip()
        author = self.author_var.get().strip()
        date = self.date_var.get().strip()
        cat = self.cat_var.get().strip() or 'Uncategorized'
        if not name:
            messagebox.showwarning("Validation", "Name is required")
```

```
            return
        # preserve id when updating
        item = self.app.data[self.index]
        item_id = item.get('id')
        # Try backend update
        try:
            resp = requests.put(f'http://127.0.0.1:5000/media/{item_id}', json={
                'name': name, 'author': author, 'date': date, 'category': cat
            }, timeout=1)
            if resp.status_code == 200:
                self.app.on_load()
                self.top.destroy()
                return
        except RequestException:
            pass
        # Fallback: update local data while preserving id
        self.app.data[self.index] = {**item, 'name': name, 'author': author, 'date': date, '
category': cat}
        self.app.write_json()
        self.app.on_load()
        self.top.destroy()
def main():
    root = tk.Tk()
    # Modernize style a bit
    try:
        style = ttk.Style()
        style.theme_use('clam')
    except Exception:
        pass
    LibraryApp(root)
    root.mainloop()
if __name__ == '__main__':
    main()
```